DEBUGGING.md — Руководство по отладке LocalScript

> Расположение в репозитории: `docs/DEBUGGING.md`

Этот документ описывает все реальные баги, с которыми мы столкнулись в ходе разработки, как они были диагностированы и решены, а также рабочие процессы для итерационной отладки модели и промптов.

Содержание

1. Выбор сценария запуска
2. Сценарий A: Docker Compose (продакшн)
3. Сценарий B: Ручной запуск (Windows, без Docker)
4. Рабочие процессы отладки промптов
5. Известные баги и их решения
6. Быстрая диагностическая шпаргалка

1. Выбор сценария запуска

Прежде чем начать — определись, что тебе нужно:

```
Хочу...                                                → Используй
──────────────────────────────────────────────────────────────────────
Запустить всё и проверить что работает                   → Сценарий A (Docker Compose)
Быстро менять код Python и перезапускать                 → Сценарий B (uvicorn --reload)
Итерировать по system_prompt.txt без API                  → Workflow 1 (Raw Ollama)
Отлаживать сборку промпта в Python                       → Workflow 2 (Python-скрипт)
Проверить полный пайплайн один запрос                    → Workflow 3 (curl / PS к /genera
Измерить точность на всём тест-сете                      → Workflow 4 (unittest runner)
```

2. Сценарий A: Docker Compose (продакшн)

Все компоненты поднимаются одной командой. Рекомендуется для финального тестирования и демонстрации. Не подходит для быстрой итерации по коду.

2.1 Первый запуск (скачать модель)

```
# Скачать модель в volume ollama-data (только один раз, ~4.7 ГБ):
docker-compose run --rm ollama ollama pull qwen2.5-coder:7b-instruct-q4_K_M
```

После этого модель хранится в Docker volume и не скачивается повторно.

2.2 Запуск

```
docker-compose up --build
```

При первом старте: сборка образа `api` (~1-2 мин) + warm-up модели (~10-15 с).

2.3 Проверка — curl (Linux / macOS / Git Bash)

```
# Базовый запрос:
curl -X POST http://localhost:8080/generate \
     -H "Content-Type: application/json" \
     -d '{"prompt": "получить последний email из списка"}'

# Ожидаемый ответ:
# {"code":"return wf.vars.emails[#wf.vars.emails]"}

# Health check:
curl http://localhost:8080/health
# {"status":"ok"}
```

2.4 Проверка — PowerShell 7+ (pwsh)

```
# PS 7+ отправляет UTF-8 по умолчанию — работает напрямую:
Invoke-RestMethod -Uri http://localhost:8080/generate `
    -Method POST `
    -ContentType "application/json; charset=utf-8" `
    -Body '{"prompt": "получить последний email из списка"}'
```

2.5 Проверка — PowerShell 5.1 (встроенный в Windows)

```
# ОБЯЗАТЕЛЬНО передавать тело как UTF-8 байты (см. Баг #1 ниже):
$json  = '{"prompt": "получить последний email из списка"}'
$bytes = [System.Text.Encoding]::UTF8.GetBytes($json)
Invoke-RestMethod -Uri http://localhost:8080/generate `
    -Method POST `
    -ContentType "application/json; charset=utf-8" `
    -Body $bytes
```

2.6 Остановка

```
docker-compose down         # остановить, сохранить volume с моделью
docker-compose down -v      # остановить и удалить volume (потребует повторного ска
```

3. Сценарий B: Ручной запуск (Windows, без Docker)

Используется при активной разработке: изменения в Python-коде видны сразу без пересборки образа.
Требует: Python 3.11+, pip, установленный Ollama CLI.

3.1 Терминал 1 — Запуск Ollama

```
# Не выгружать модель из VRAM при простое (иначе каждый запрос будет ждать загрузки):
$env:OLLAMA_KEEP_ALIVE = "-1"
ollama serve
# Оставить этот терминал открытым!
```

> **Важно:** Запускай Ollama через CLI (`ollama serve`), а не через GUI-приложение.
> GUI не позволяет задать переменные среды.

3.2 Проверка Ollama (любой другой терминал)

```
# Проверить что Ollama отвечает (простой английский запрос — PS 5.1 и так справится):
Invoke-RestMethod -Uri http://localhost:11434/api/generate -Method POST `
    -ContentType "application/json" `
    -Body '{"model":"qwen2.5-coder:7b-instruct-q4_K_M","prompt":"hello","stream":f

# Ожидаемый ответ: объект с полем response = "Hello! How can I assist you today?"
```

3.3 Терминал 2 — Запуск uvicorn

```
# Активировать виртуальное окружение:
.venv\Scripts\activate

# Настроить переменные среды:
$env:NO_PROXY        = "localhost,127.0.0.1,::1"  # обойти системный прокси
$env:DRY_RUN         = "true"                     # пропустить luac (нет на Windows)
$env:OLLAMA_BASE_URL = "http://localhost:11434"
$env:OLLAMA_MODEL    = "qwen2.5-coder:7b-instruct-q4_K_M"
$env:MAX_RETRIES     = "3"

# Запустить сервер с hot-reload:
uvicorn api.main:app --reload --port 8080

# При первом старте warm-up займёт ~10-15 секунд (модель загружается в VRAM).
# Ждать строки: Application startup complete.
```

3.4 Терминал 3 — Тестовые запросы

Через curl (Git Bash / WSL — рекомендуется, всегда UTF-8):

```
curl -X POST http://localhost:8080/generate \
     -H "Content-Type: application/json" \
     -d '{"prompt": "отфильтровать заказы со статусом выполнен"}'
```

Через PowerShell 5.1 (с явной UTF-8 кодировкой):

```
$json  = '{"prompt": "отфильтровать заказы со статусом выполнен"}'
$bytes = [System.Text.Encoding]::UTF8.GetBytes($json)
Invoke-RestMethod -Uri http://localhost:8080/generate `
    -Method POST -ContentType "application/json; charset=utf-8" -Body $bytes
```

Через PowerShell 5.1 — с файлом (альтернатива):

```
'{"prompt": "отфильтровать заказы со статусом выполнен"}' | `
    Out-File -Encoding utf8NoBOM body.json

Invoke-RestMethod -Uri http://localhost:8080/generate `
    -Method POST -ContentType "application/json; charset=utf-8" -InFile body.json
```

Через Python (самый надёжный способ):

```
import httpx
r = httpx.post("http://localhost:8080/generate",
              json={"prompt": "отфильтровать заказы со статусом выполнен"})
print(r.json()["code"])
```

4. Рабочие процессы отладки промптов

Workflow 1 — Raw Ollama (обход всего Python-кода)

Когда использовать: хочешь проверить, как модель реагирует на промпт, не меняя Python-код. Самая быстрая петля итерации.

```
# Сформировать промпт вручную и отправить напрямую в Ollama:
$prompt = @"
You are a Lua code generator for MWS Octapi platform.

<thinking>
1. Translate task to English
2. Plan the Lua logic step by step
</thinking>
<code>
[pure Lua here]
</code>
```

```
USER TASK: filter orders where status equals done
"@

$body = @{
    model  = "qwen2.5-coder:7b-instruct-q4_K_M"
    prompt = $prompt
    stream = $false
} | ConvertTo-Json -Depth 5

$bytes = [System.Text.Encoding]::UTF8.GetBytes($body)
$r = Invoke-RestMethod -Uri http://localhost:11434/api/generate `
    -Method POST -ContentType "application/json; charset=utf-8" -Body $bytes
Write-Host $r.response
```

Или через Python — удобнее для длинных промптов:

```
import httpx

system_prompt = open("prompts/system_prompt.txt", encoding="utf-8").read()
user_task = "отфильтровать заказы со статусом выполнен"
full_prompt = f"{system_prompt}\n\nUSER TASK: {user_task}"

r = httpx.post("http://localhost:11434/api/generate", json={
    "model": "qwen2.5-coder:7b-instruct-q4_K_M",
    "prompt": full_prompt,
    "stream": False
}, timeout=60)
print(r.json()["response"])
```

Workflow 2 — Python-скрипт (тест PromptBuilder + Ollama)

Когда использовать: хочешь проверить, как PromptBuilder собирает финальный промпт, без запуска uvicorn. Позволяет видеть и изменять сборку промпта в Python-коде.

```
# debug_prompt.py — запустить из корня репозитория
import sys, os
sys.path.insert(0, ".")
os.environ["OLLAMA_BASE_URL"] = "http://localhost:11434"
os.environ["OLLAMA_MODEL"]    = "qwen2.5-coder:7b-instruct-q4_K_M"
os.environ["DRY_RUN"]         = "true"

from api.prompt_builder import PromptBuilder
from api.agent import OllamaClient

builder = PromptBuilder()
client  = OllamaClient()

user_prompt = "отфильтровать заказы со статусом выполнен"

# Посмотреть финальный промпт перед отправкой:
full_prompt = builder.build(user_prompt)
```

```
print("=== ФИНАЛЬНЫЙ ПРОМПТ ===")
print(full_prompt[:2000])  # первые 2000 символов

# Отправить и получить ответ:
print("\n=== ОТВЕТ МОДЕЛИ (RAW) ===")
raw = client.generate(full_prompt)
print(raw)
```

Запустить:

```
$env:NO_PROXY = "localhost,127.0.0.1,::1"
python debug_prompt.py
```

Workflow 3 — **Полный пайплайн через API (один запрос)**

Когда использовать: uvicorn запущен, хочешь проверить конкретный запрос через весь стек: PromptBuilder → OllamaClient → CoT-парсер → LuaValidator.

```
# curl (Git Bash / WSL):
curl -sX POST http://localhost:8080/generate \
     -H "Content-Type: application/json" \
     -d '{"prompt": "отфильтровать заказы со статусом выполнен"}' | python -m json.t
```

```
# Python-скрипт:
import httpx
tasks = [
    "отфильтровать заказы со статусом выполнен",
    "оставить только выполненные заказы",
    "убрать незавершённые заявки",
]
for task in tasks:
    r = httpx.post("http://localhost:8080/generate",
                   json={"prompt": task}, timeout=60)
    print(f"Task: {task}")
    print(f"Code: {r.json().get('code', 'ERROR')}\n")
```

Workflow 4 — **Полный тест-сет (измерение точности)**

Когда использовать: после изменения `system_prompt.txt` — для количественной оценки влияния изменений на точность.

```
# Убедись что API запущен (Docker или uvicorn), затем:
python -m unittest tests.test_base_cases.TestBaseCases

# Отчёт сохраняется в tests/reports/report_YYYYMMDD_HHMMSS.{md,json}
```

Для просмотра последнего отчёта:

```
Get-ChildItem tests/reports/ | Sort-Object LastWriteTime -Descending | Select-Objec
```

5. Известные баги и их решения

Баг #1 — PowerShell 5.1: кириллица становится `????????`

Симптом:

```
PS> Invoke-RestMethod ... -Body '{"prompt":"Привет, ты понимаешь русский?"}'

response : I apologize, but I'm not able to understand or respond to the message yo
           It appears to be in a language I don't recognize.
```

Или, ещё хуже, модель «переводит» `???????` как что-то совершенно случайное:

```
response : The translation of "???????, ?? ????????? ?????????" is:
           "Look, how the stars are shining in the sky"
```

Причина:

PowerShell 5.1 (встроенный в Windows 10/11) кодирует строку в параметре `-Body` как ASCII. Каждый символ вне ASCII-диапазона (U+0000–U+007F) заменяется на `?`. Кириллица полностью в этом диапазоне не помещается — все символы теряются.

Это не баг модели. Qwen2.5-Coder обучена на 29+ языках и прекрасно понимает русский, если получает его в правильной кодировке.

Диагностика:

Проверь версию PowerShell:

```
$PSVersionTable.PSVersion
# Major 5 → проблема с кодировкой
# Major 7 → UTF-8 по умолчанию, проблем нет
```

Решение 1 — UTF-8 байты (рекомендуется для PS 5.1):

```
$json  = '{"model":"qwen2.5-coder:7b-instruct-q4_K_M","prompt":"Привет, ты понимаеш
$bytes = [System.Text.Encoding]::UTF8.GetBytes($json)
Invoke-RestMethod -Uri http://localhost:11434/api/generate `
    -Method POST -ContentType "application/json; charset=utf-8" -Body $bytes
```

Решение 2 — файл с явной кодировкой utf8NoBOM:

```
'{"prompt": "отфильтровать заказы"}' | Out-File -Encoding utf8NoBOM body.json
Invoke-RestMethod -Uri http://localhost:8080/generate `
    -Method POST -ContentType "application/json; charset=utf-8" -InFile body.json
```

Решение 3 — использовать PowerShell 7+:

```
# Установить: winget install Microsoft.PowerShell
# Запустить: pwsh (не powershell)
pwsh -Command 'Invoke-RestMethod -Uri http://localhost:8080/generate -Method POST -C
```

Решение 4 — curl (всегда работает, UTF-8 по умолчанию):

```
curl -X POST http://localhost:8080/generate \
     -H "Content-Type: application/json" \
     -d '{"prompt": "отфильтровать заказы"}'
```

Влияние на продакшн: нулевое. Python `httpx` всегда отправляет UTF-8.
Проблема касается только инструментов ручной отладки на Windows.

Баг #2 — 502 Bad Gateway при запросах к localhost через Ollama / uvicorn

Симптом:

```
httpx.ProxyError: 502 Bad Gateway via proxy
```

или

```
[WinError 10061] Подключение не установлено
```

Причина:

Системный прокси-сервер (Clash Verge, Proxifier, корпоративный VPN-клиент и т.д.)
перехватывает ВСЕ HTTP-запросы Python, включая запросы к `localhost`.
Прокси не умеет маршрутизировать loopback-адреса и возвращает ошибку.

PowerShell при этом работает нормально — он использует .NET `HttpClient`,
который по умолчанию исключает `localhost` из прокси.

Диагностика:

```
# Работает (PS не через прокси):
Invoke-RestMethod -Uri http://localhost:11434/api/generate ...

# Не работает (Python через прокси):
python -c "import httpx; print(httpx.get('http://localhost:11434').text)"
```

Решение — добавить NO_PROXY перед запуском uvicorn:

```
$env:NO_PROXY = "localhost,127.0.0.1,::1"
uvicorn api.main:app --reload --port 8080
```

Или добавить в системные переменные среды раз и навсегда:
`Системные свойства → Переменные среды → Системные переменные → Создать`:

- Имя: `NO_PROXY`
- Значение: `localhost,127.0.0.1,::1`

Влияние на Docker: нулевое. Контейнеры общаются через внутреннюю Docker-сеть, а не через системный прокси Windows.

Баг #3 — `ModuleNotFoundError: No module named 'test_base_cases_data'`

Симптом:

```
ImportError: Failed to import test module: test_base_cases
...
ModuleNotFoundError: No module named 'test_base_cases_data'
```

Причина:

Файл был переименован из `test_base_case_data.py` в `test_base_cases_data.py`.
При запуске через PyCharm (gutter icon) или `python -m unittest tests.test_base_cases.TestBaseCases`
Python ищет модули относительно **корня проекта**, а не папки `tests/`.
Импорт `from test_base_cases_data import TEST_CASES` не находит модуль, потому что ищет его в корне, а не в `tests/`.

Решение — использовать абсолютный импорт:

В файле `tests/test_base_cases.py` заменить:

```
# Неправильно:
from test_base_cases_data import TEST_CASES

# Правильно:
from tests.test_base_cases_data import TEST_CASES
```

Баг #4 — Модель выгружается из VRAM после 5 минут простоя

Симптом:

Первый запрос после паузы выполняется 15-20 секунд вместо обычных 1-3 секунд.
В логах Ollama: `model unloaded`.

Причина:

По умолчанию Ollama выгружает модель из VRAM через 5 минут без запросов для освобождения памяти.

Решение для локальной разработки:

```
$env:OLLAMA_KEEP_ALIVE = "-1"
ollama serve
```

Значение `-1` означает «никогда не выгружать». Значение `0` — выгрузить сразу после ответа.

В Docker Compose: модель загружается в VRAM при warm-up и остаётся там, пока контейнер запущен. В `docker-compose.yml` можно добавить:

```
environment:
  - OLLAMA_KEEP_ALIVE=-1
```

Баг #5 — DRY_RUN=false на Windows: `luac: command not found`

Симптом:

```
FileNotFoundError: [WinError 2] Не удается найти указанный файл: 'luac'
```

Причина:

`luac` (компилятор Lua, используемый для синтаксической проверки) не входит в стандартную поставку Windows. При `DRY_RUN=false` `validator.py` пытается его запустить.

Решение:

```
$env:DRY_RUN = "true"
```

При `DRY_RUN=true` валидатор пропускает проверку и всегда возвращает `valid=True`.
Retry-loop всё ещё работает, но не по причине синтаксических ошибок.

В Docker-контейнере `lua5.4` установлен через `apt-get` в Dockerfile, и `DRY_RUN=false` работает корректно.

6. Быстрая диагностическая шпаргалка

```
Проблема                                        Вероятная причина          Раздел
──────────────────────────────────────────────────────────────────────────────────
Модель "не понимает русский"                →   Баг #1 (PS 5.1 + ASCII)    §5.1
502 Bad Gateway при запросах Python         →   Баг #2 (системный прокси)  §5.2
ModuleNotFoundError test_base_cases_data    →   Баг #3 (импорт)           §5.3
Первый запрос после паузы очень медленный   →   Баг #4 (KEEP_ALIVE)       §5.4
luac: command not found на Windows          →   Баг #5 (DRY_RUN)          §5.5
Ollama не отвечает изнутри Docker           →   Адрес должен быть ollama:11434, не loc
API отвечает, но код пустой                 →   CoT-парсер не нашёл тег <code>
                                                Проверь system_prompt.txt
Warm-up занимает больше 30 секунд           →   Модель не скачана или повреждена
                                                docker-compose run --rm ollama ollama l
```